

Anthony Martuscelli

This week's work was to extend the parking app so chargers are computed based on new rules and set rates. In the first release, there would have been fixed rates that only changed based on vehicle type. With the business thought of more attractive pricing, differentiating rates were added into the system. This meant pricing algorithms had to be easy to replace, and each of the parking lots might use a different one.

For simplicity, I went with factors that the system already measured and created the following rules:

- Parking costs \$2 per hour, with partial hours rounded up.
- Charges are capped at \$15 per calendar day.
- Sunday parking is free.
- Compact cars receive a 20% discount, applied after the normal charge and daily cap. SUVs pay the regular charge.

The `ParkingChargeStrategy` interface lives in the `charges.strategy` package. Its calculation method receives the lot's base rate, the entry time, the exit time, and the `ParkingPermit`, and returns a `Money`. The permit leads back to the `Car` and `Customer`, so a strategy can inspect vehicle type without extra parameters.

The concrete strategies include

HourlyParkingChargeStrategy - Applies hourly rate, rounds partial hours up, and enforces daily max

SundayFreeParkingChargeStrategy - Charges nothing on Sundays

CompactDiscountParkingChargeStrategy - Sets the base calculation, then applies a 20% discount only when the permit belongs to a compact car.

The Sunday and discount strategies wrap other strategies and compose layers. The discounts know nothing of the hourly or Sunday pricing.

So as I worked to find concepts of different strategies to make parking more appealing, I found each almost opened the door to a new set of challenges or added layers of complexity. Some came easier than others, but some grew very technical in nature.

First was avoiding a monolithic calculator. One easy option would have been to put all parking rules into one class with a lot of conditions. While this could have worked, maintenance would have been difficult, especially as the app grew and others might be supporting it. Using separate strategies with a shared interface kept each rule smaller and a lot easier to test. Downside, it created more classes. In the end I felt this was reasonable in the design.

Another was multiday parking. Handling parking that crosses multiple days was a more complicated part. The daily maximum applies to each calendar day, but I decided to go with free Sunday's for the appeal of it. So a stay from Saturday evening to Monday morning cannot simply be treated as one long period. I divided the parking time into separate calendar-day sections and applied the appropriate rules to each day.

I also had to be cautious about where discounts were applied. Applying the discount before the daily maximum could produce a different result than applying it after the normal charge was calculated. I used a wrapper strategy so that the regular parking charge was calculated first, and the discount was applied afterward. I also added a test just for this situation to ensure the order was correct.

I used BigDecimal through a Money classes instead of relying on standard float pointing numbers. These can end up producing small rounding errors when working with things like money amounts. Since this system is calculating parking charges, I wanted the money calculations to be accurate and not floating.

One thing that actually really set me back was just a simple mistake I made while reviewing requirements. For some reason, I coded up to give the discount to SUV's rather than compacts. While the correction was really straightforward, I didn't even realize what I did until I was about to push to GitHub. This was pretty simple to correct but was an indication to follow order of doing things.

Overall, I was pleased with what I delivered. This week showed that what seems like adding simple steps, can turn into a lot of complexity pushed into an existing system. The strategy pattern gave me a way to separate these rules and make them easier to change without having to rewrite a large portion of the application. I also learned that the design is only part of the process. Testing singular rules, thinking about edge cases, and following requirements are really important steps.